

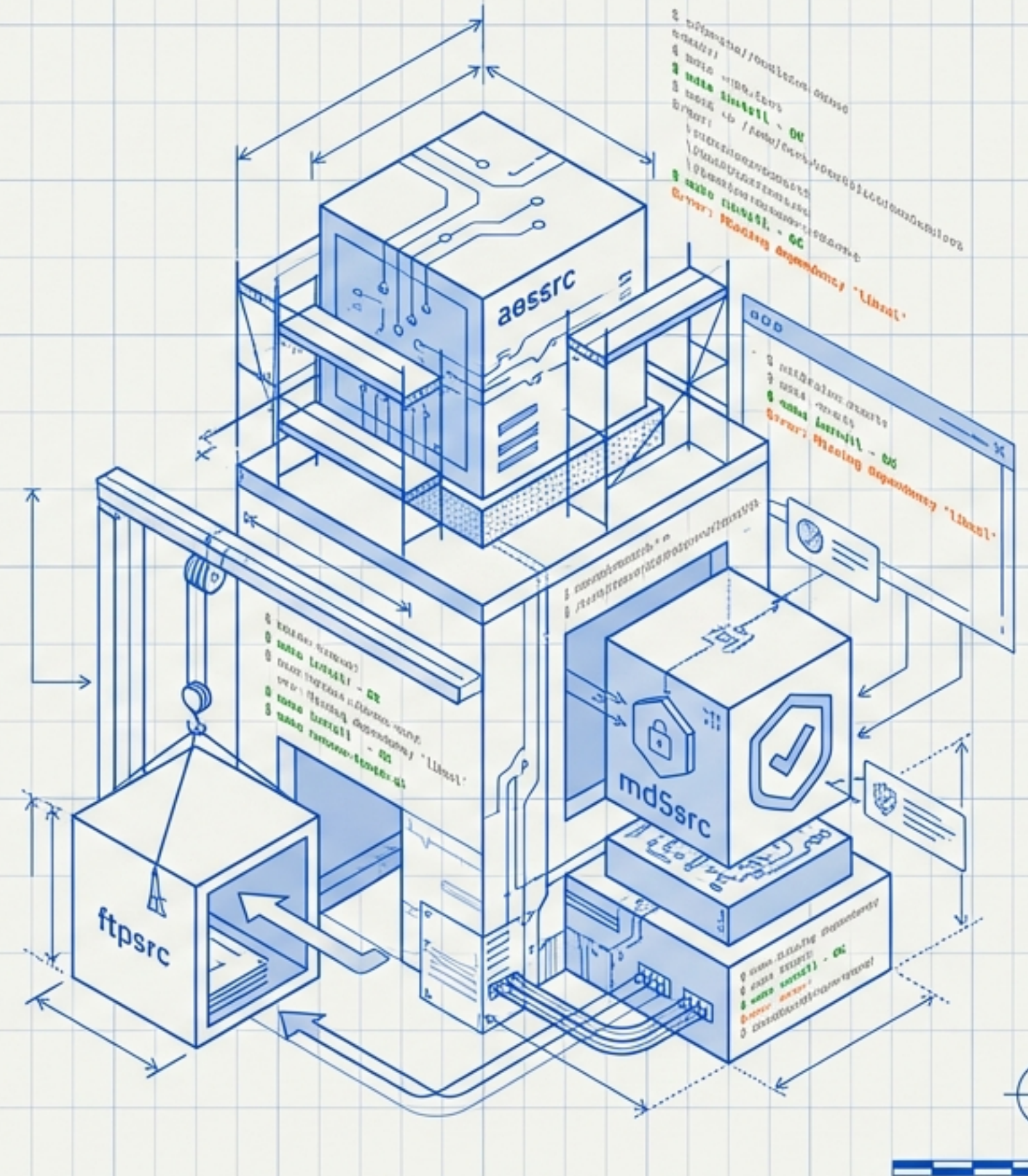
ECAMS Agent 빌드 전략 및 트러블슈팅 가이드

UNIX/Linux 크로스 플랫폼 컴파일 완벽 분석

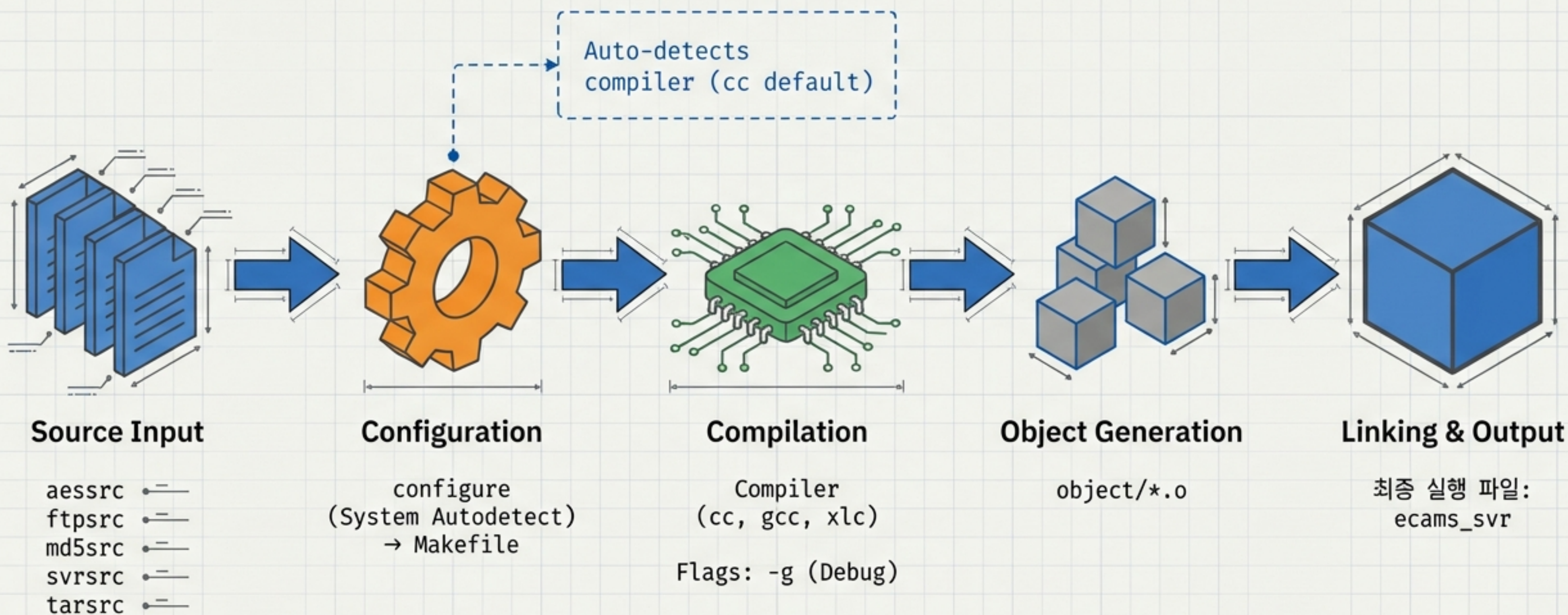
Target System: ecams_svr Agent

Scope: Solaris, AIX, HP-UX, Linux (RHEL, Alpine, Embedded)

Objective: From compilation errors to automated deployment

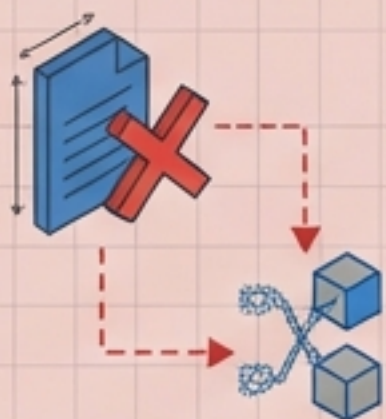


빌드 시스템 아키텍처



공통 컴파일 오류 I: 환경 및 의존성

Header Not Found / 헤더 파일 없음



Symptom:

네트워크 관련 개발 헤더 미설치

Error:

fatal error: sys/socket.h:
No such file or directory



Solution / 해결책



Rx (Linux):

Debian/Ubuntu, RHEL/CentOS
패키지 설치 확인

Rx (Solaris):

시스템 헤더 경로 확인

Type Not Defined / 타입 정의 누락



Symptom:

uint 등 일부 타입 미정의
(Legacy Systems)

Error:

error: unknown type name
'uint'



Solution / 해결책



Rx 1:

configure.ac 내
PSG_REPLACE_TYPE 체크 로직 확인

Rx 2:

config.h 확인 (권장)
또는 소스 코드 수정

공통 컴파일 오류 II: 코드 호환성

Implicit Declaration (함수 선언 누락)

Diagnosis: autoscan.log에
누락된 함수 체크 내역 존재.

⚠ warning: implicit declaration of function 'foo'

✅ **Solution:** 누락된 헤더 파일 추가
및 configure.ac 체크 로직 보완.

Redefinition (중복 정의)

Diagnosis: 시스템 라이브러리와
사용자 헤더 간 심볼 충돌.

```
#ifndef MY_HEADER_H
#define MY_HEADER_H
...
#endif
```

✅ **Solution:** 헤더 가드(Header
Guard) 강화 및 호환성 함수 조건부
컴파일 적용.

Makefile Variable Undefined

Cause: Makefile.in이 Makefile로
변환되지 않음 (./configure 미실행).



✅ **Fix:** Run ./configure first.

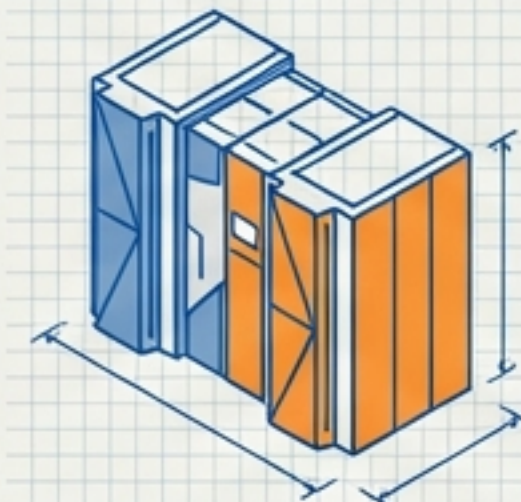
OS별 특화 이슈 I: Enterprise UNIX (Solaris & AIX)

Solaris



- ✓ Library Link: 네트워크 함수 분리 이슈
→ `configure.ac`에서 `-lsocket` 등 라이브러리 자동 감지 설정
- ✓ FS Compatibility: `statvfs` vs `statfs` 차이 해결
- ✓ Path Priority: `/usr/xpg4/bin` 우선순위 확인

AIX



- ✓ Compiler: `xlc` 컴파일러 고유 경고(Warning) 처리
- ✓ Architecture: 32bit vs 64bit 아키텍처 빌드 옵션 결정

OS별 특화 이슈 II: HP-UX 및 Linux 변형

HP-UX



Standard: ANSI C 모드 준수 필요

Network: 네트워크 라이브러리 링크 확인

Linux Variants

Alpine Linux

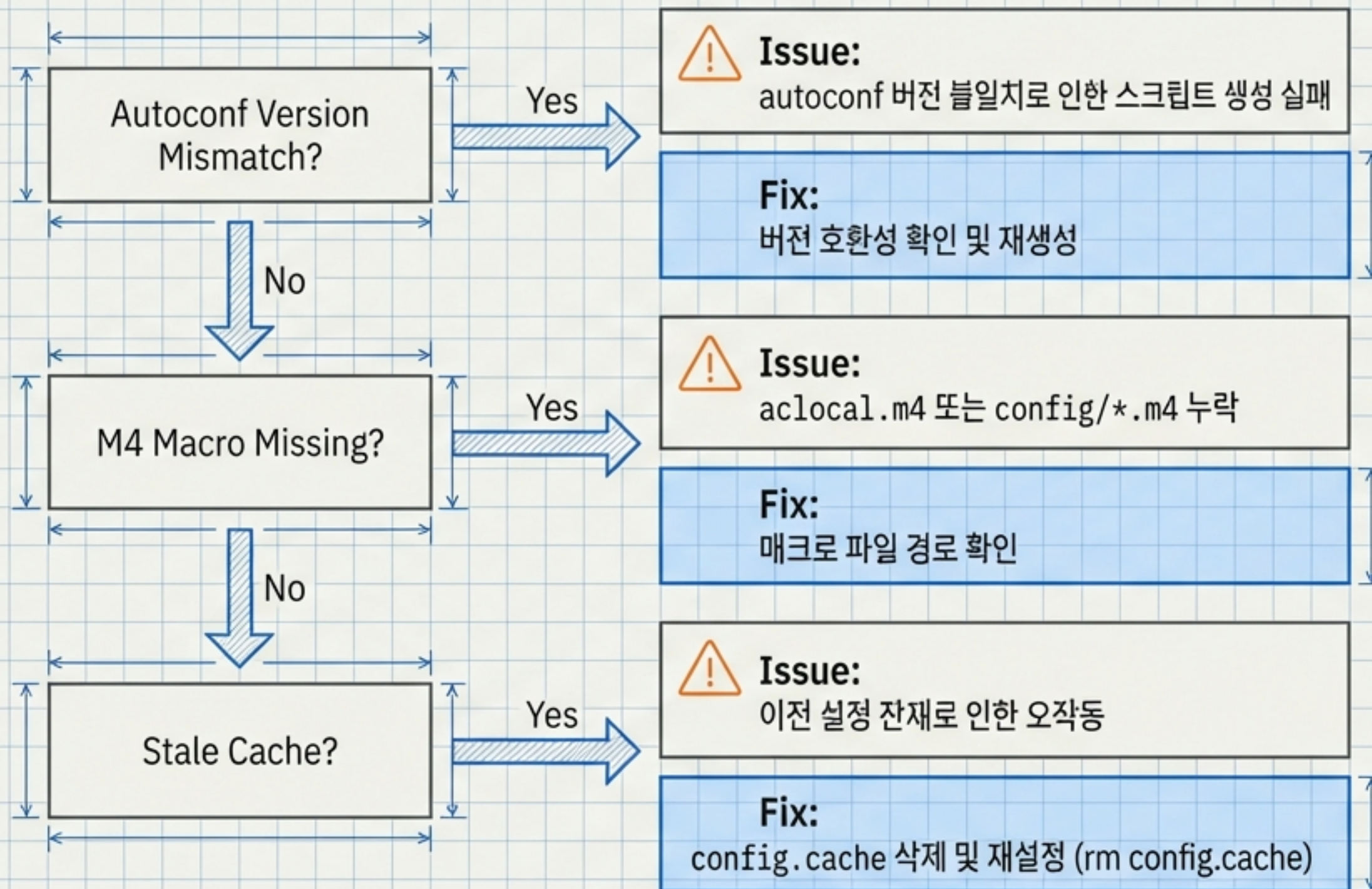
musl libc 사용에 따른 호환성 처리



Embedded Linux (uclibc)

불필요한 crypt 함수 제거 또는
-lcrypt 명시적 추가

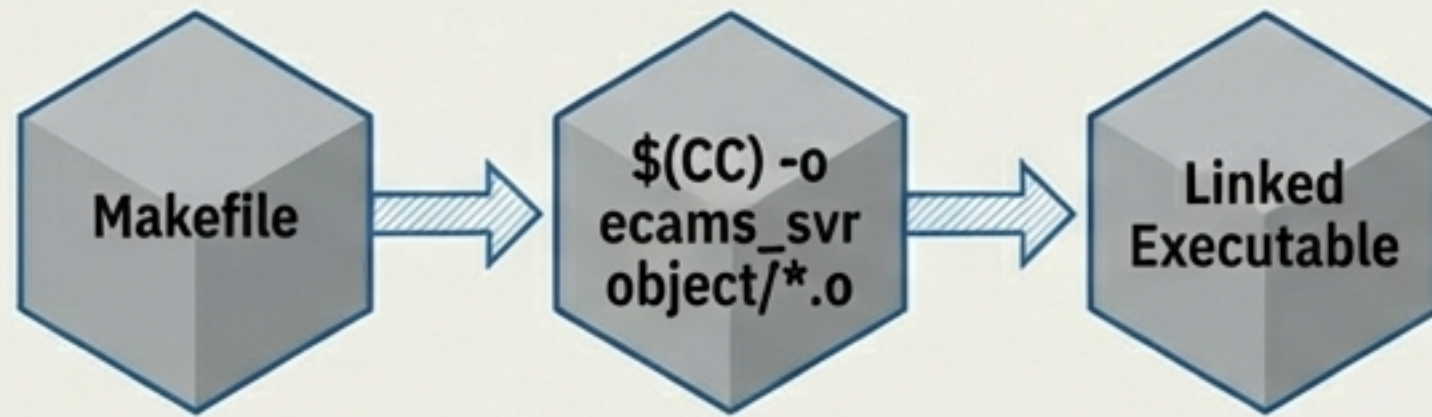
Configure 스크립트 트러블슈팅



링커(Linker) 오류 및 해결

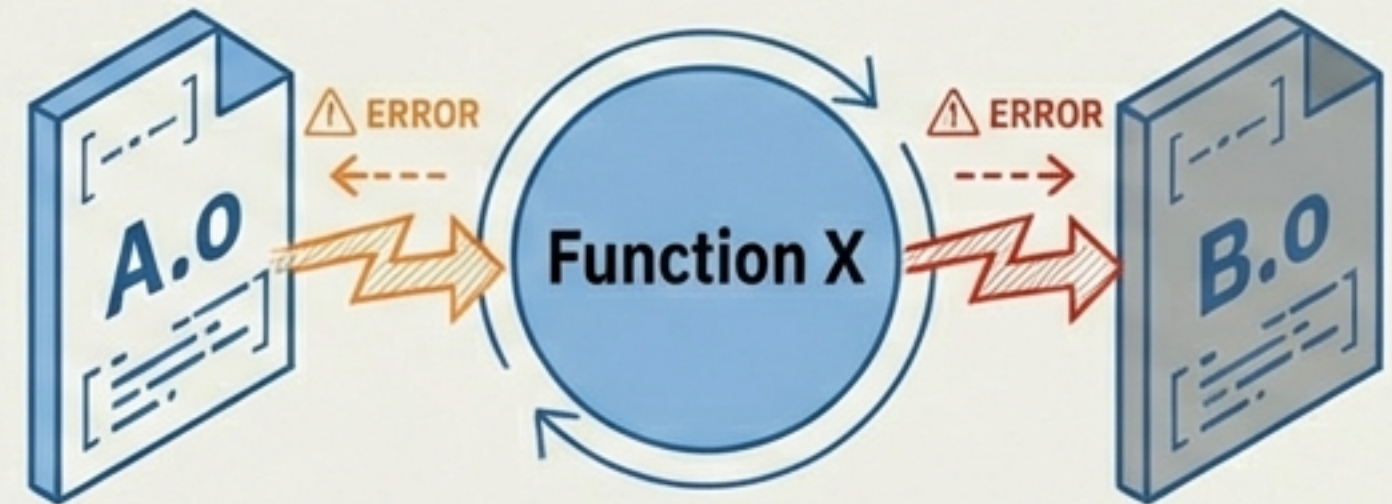
Key Issue 1: Library Order (라이브러리 순서)

Compiler Process



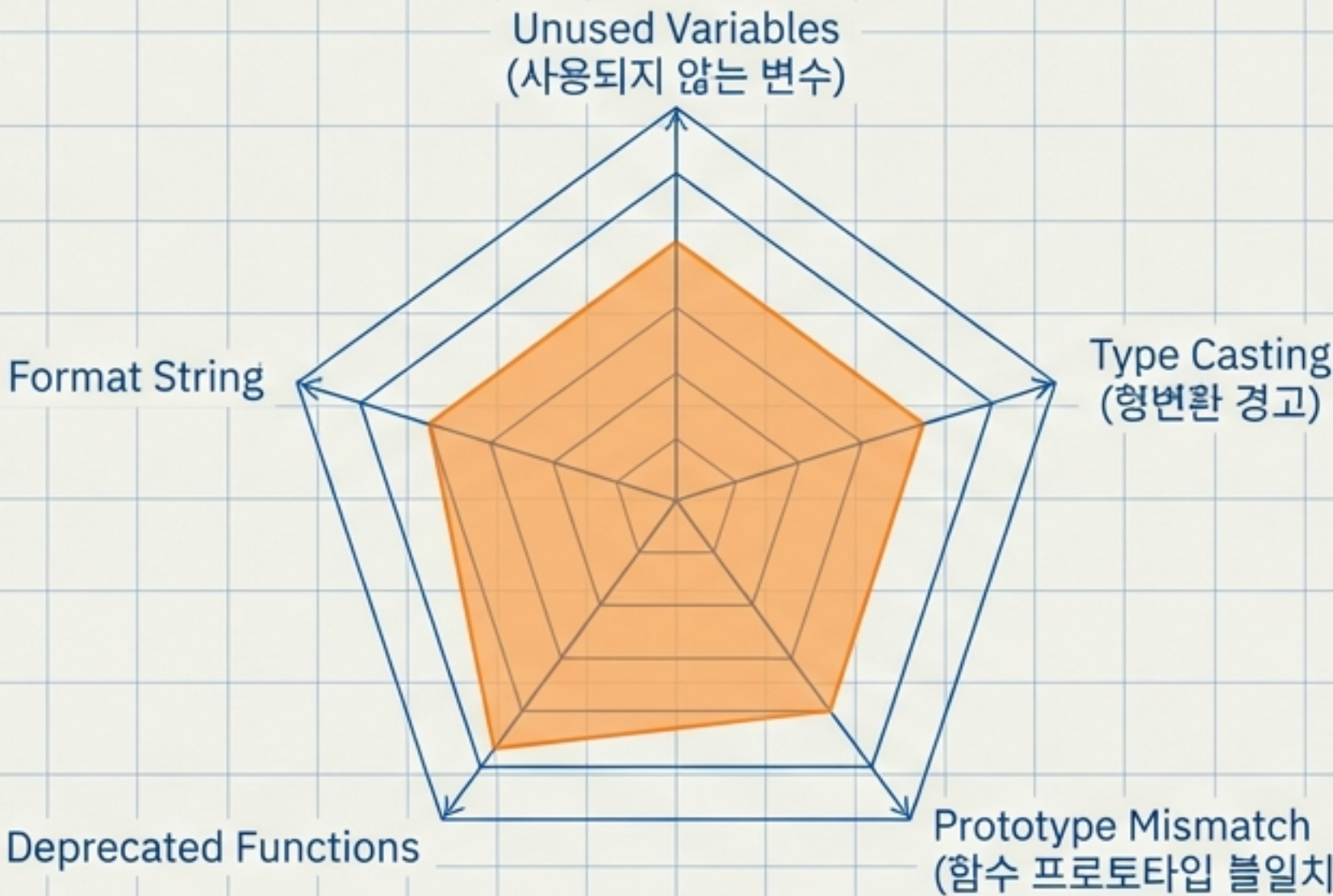
Cause: Makefile 내 \$(CC) -o ecams_svr object/*.o 순서 문제
Fix: 의존성 순서에 맞춰 오브젝트 파일 배열 (Dependent .o last)

Key Issue 2: Duplicate Symbols (중복 심볼)



Cause: 동일 함수가 여러 .o 파일에 존재
Fix: 중복 정의된 소스 코드 정리 및 extern 처리

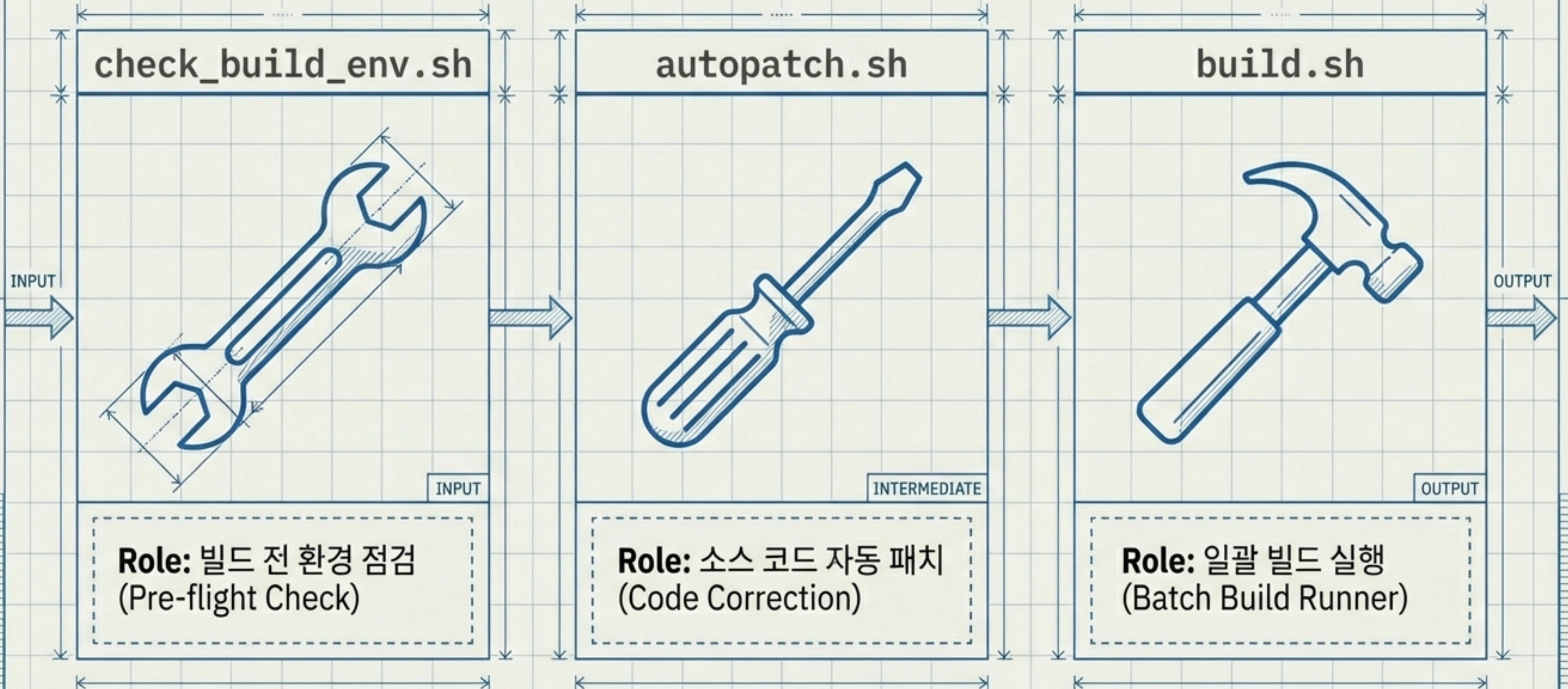
Warning 관리 및 코드 품질



- Unused Variables: 불필요한 메모리 사용 및 로직 오류 가능성
- Type Casting: 포인터 및 데이터 손실 위험
- Prototype Mismatch: 호출 규약 오류
- Deprecated Functions: 구식 함수 사용 지양
- Format String: 보안 취약점 점검

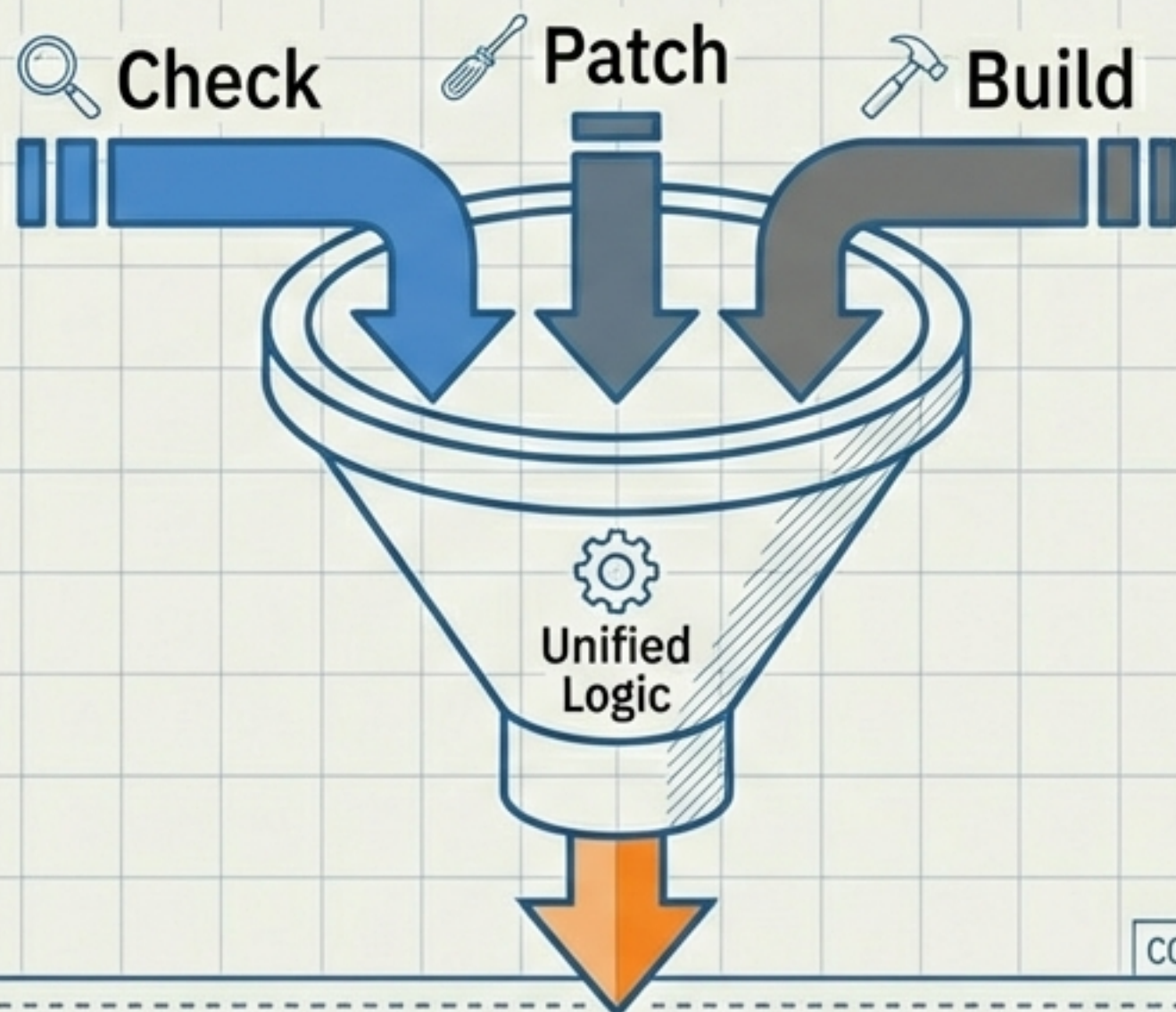
Strategic Goal:
Clean Build (Warning 0)

자동화 도구 모음 (Automation Toolkit)



통합 일괄 빌드 솔루션

The Game Changer



Core Concept

3단계 프로세스(Check → Patch → Build)를 단일 명령으로 통합

Workflow

1. 환경 점검 수행
2. 필요 시 자동 패치 적용
3. 빌드 및 로그 생성

Result

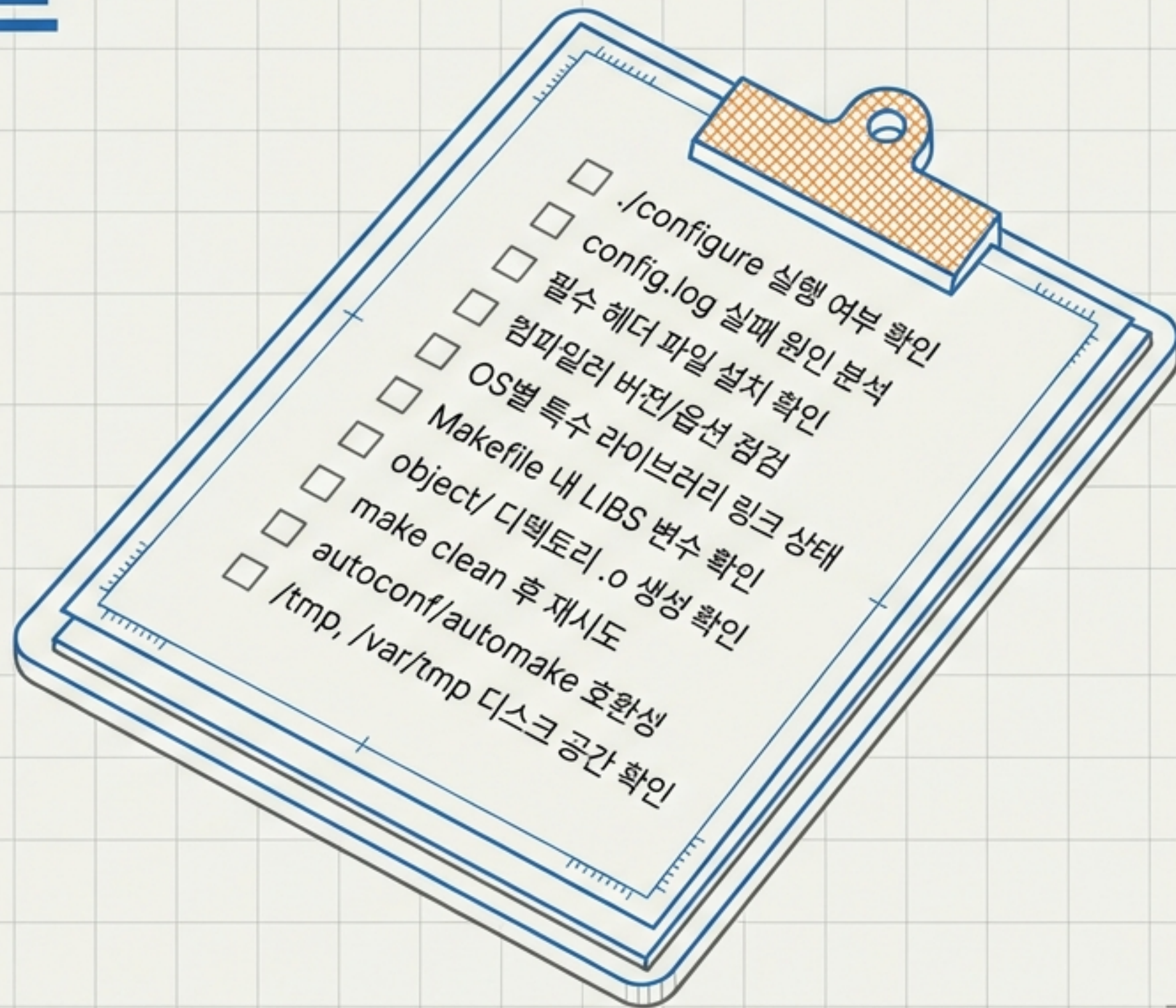
단일 스크립트 실행으로 전체 프로세스 완료

기존 방식 vs 통합 방식 효율성 비교

항목 (Category)	기존 방식 (Legacy)	통합 방식 (New build_all.sh)
실행 명령	./check && ./patch && ./build ↗	./build_all.sh (1회) ⚙️
스크립트 수	3개	1개
로그 관리	개별 로그 산재	통합 로그 자동 생성
옵션 지원	없음	--skip-check, --skip-patch 등
에러 처리	수동 확인 필요 ⚠️	단계별 자동 중단 (Fail-fast) ✅ →

트러블슈팅 체크리스트

Actionable Steps for Debugging



심층 디버깅 및 리소스

Analyze config.log



Configure 단계의 실패 원인을
찾는 가장 확실한 방법.
Search for 'error' or 'failed'.

Log Retention



에러 로그 별도 저장 및 분석.

Detailed Debugging



컴파일러 상세 옵션 활용 및
시스템 공간 확인 (/tmp).

문서 정보 및 결론

체계적인 환경 점검과 통합 스크립트 활용이
성공적인 빌드의 핵심입니다.

Version

1.0
JetBrains Mono

Date

2026-01-23

Author

Polaris (Senior Architect)

Basis

configure.ac, autoscan.log,
Makefile 분석 기반